异构协同的低空无人机检测系统

技术文档

摘要

本作品面向固定点位的低空无人机监测，基于复旦微 FMQL30TAI 悟净开发板（完整版），实现了相机采集、无人机检测、目标编号、区域告警和画面展示。接好相机并通电后，板卡自行检测和告警，同时制作了配套上位机用于查看结果与操作。

异构协同方面，CPU 负责采集、图像预处理、结果整理和通信，NPU 运行模型，FPGA 的 PL 端根据目标位置和连续检测结果控制告警，同时独立监测输入超时、驱动故障灯并记录事件发生时间。检测与硬件告警均在板端完成。

本作品的模型在 1629 张已用于开发的图像上取得 INT8 精确率 95.65%、召回率 95.77%，INT8 量化仿真与 NPU 实板输出框逐框一致。完整软硬件版本通过一小时相机运行，平均 29.83 FPS。另完成独立 IR 模型实板推理和 FPGA 图像预处理验证，作为扩展保留。

关键词：无人机检测；异构协同；边缘计算

目录

第一部分　作品概述 4
1.1 功能与特性 4
1.2 应用领域 4
1.3 主要技术特点 4
1.4 主要性能指标 4
1.5 主要创新点 5
1.6 设计流程 5
第二部分　系统组成及功能说明 6
2.1 整体介绍 6
2.2 硬件系统介绍 7
2.2.1 硬件整体介绍 7
2.2.2 机械设计介绍 7
2.2.3 电路及逻辑模块介绍 7
2.3 软件系统介绍 9
2.3.1 软件整体介绍 9
2.3.2 软件各模块介绍 9
2.3.3 团队与 AI agent 协作开发 12
第三部分　完成情况及性能参数 16
3.1 整体介绍 16
3.2 工程成果 16
3.2.1 机械成果 16
3.2.2 电路成果 16
3.2.3 软件成果 16
3.2.4 保留扩展的完成情况 17
3.3 特性成果及量化测试 17
3.3.1 模型检测精度与部署一致性 17
3.3.2 告警一致性、连续确认与故障响应 17
3.3.3 运行速度与优化对照 18
3.3.4 启动、停止与一小时稳定性测试 19
3.3.5 FPGA 资源占用与时序结果 20
3.3.6 实板演示与现场测试 20
3.3.7 保留扩展的验证结果 21
第四部分　总结 23
4.1 可扩展之处 23
4.2 心得体会 23
第五部分　参考文献 24

赛题评分要求阅读索引

关键文件位于 GitHub 材料或其中的 01_source/core_source 代码目录；完整路径见 README 及代码辅助阅读页面。

评分项目	评分内容	作品对应内容	文档位置
方案设计	场景选择与需求分析	应用场景与作品功能 对应文件：README.md	1.1、1.2、1.5
方案设计	系统架构与软硬件划分	数据流与 CPU、NPU、FPGA 分工 对应文件：worker.py；product_stream.cpp；region_alarm_core.v	1.3、2.1、2.2、2.3
方案实现	AI 模型部署与工具链使用	数据准备、量化转换与部署验证 对应文件：DATASET_SOURCES.md；export_raw_heads_pinned.py	2.3.2、3.3.1
方案实现	FPGA 模块开发与系统集成	帧事务、区域告警、超时监督与集成 对应文件：src/pl/alarm/；tests/pl/	2.2.3、3.2.2、3.3.2
方案实现	功能完整性与稳定性	输入、检测、告警、界面及持续运行 对应文件：run_camera.py；Desktop.cs；camera_1h_final_result.json	2.3.2、3.2、3.3.4、3.3.6
性能表现	精度、速度与资源分析	检测指标、运行对照、资源及功耗说明 对应文件：实测结果.md；model11_results.json	1.4、3.3.1、3.3.3—3.3.5
作品成果	文档、代码、答辩与团队协作	工程成果与协作；代码和演示材料配套提交 对应文件：README.md；01_source/README.md	2.3.3、3.1、3.2
附加分	AI 工具效率提升表现	AI 参与、工作量估计与人工核验 对应文件：llm_usage.md；代码辅助阅读.html	2.3.3、4.2

第一部分　作品概述

1.1 功能与特性

本作品使用固定 USB 彩色相机监测画面中的无人机，显示检测框、置信度和目标编号。用户可设置 ROI 关注区域；目标进入该区域后，系统结合连续检测结果控制实体告警灯，并在上位机显示画面与状态。

板端支持通电自启、持续检测和正常停止。LED2 显示目标告警，LED1 显示检测结果超时；Windows 上位机提供认证启停、暂停预览、保存画面和记录导出。检测与告警由板卡独立运行，电脑用于查看和操作。

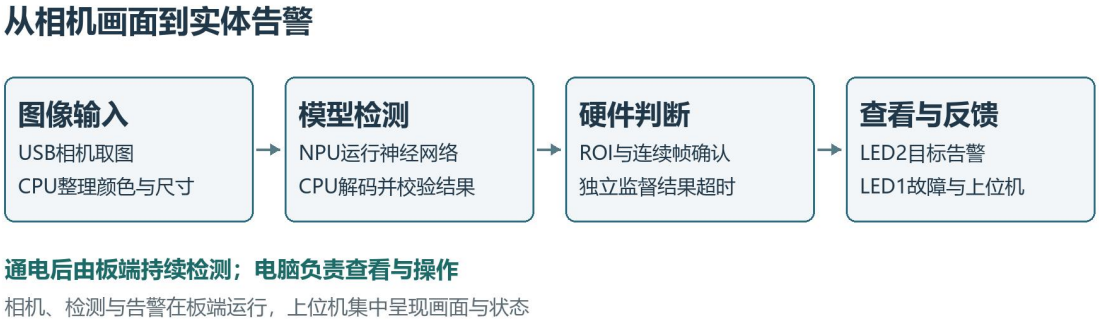


图 1　作品功能与交互流程

1.2 应用领域

作品面向固定点位的低空可视辅助监测：在工业园区和仓储基地，观察围墙、屋顶周边的可见空域；在变电站、供水设施等重要基础设施周边，为值守人员提供目标画面和区域告警；在会展、体育活动外围，通过临时观察点辅助人工确认定向目标等等。

1.3 主要技术特点

CPU 负责图像准备、结果整理与通信，NPU 运行模型，FPGA 维护区域告警和超时状态，通过完整帧事务交接。软件合并图像操作与推理提交，并行校验输入，以有界队列处理预览和录像；硬件使用独立时基记录事件并驱动双灯。

1.4 主要性能指标

项目	结果
相机 / 模型输入	640×480 / 640×640
INT8 开发验证	P(精确率) 95.65% / R(召回率) 95.77% / F1 95.71%

项目	结果
连续一小时运行	107396 次推理 / 107396 次提交；29.83 FPS
告警 事件记录与时基	270 LUT / 830 FF；100 MHz

1.5 主要创新点

1）将 CPU 图像处理、NPU 模型推理和 FPGA 独立告警组织为单板检测系统，离线时仍能检测与反馈。

2）并行校验与推理，合并图像操作及 SDK 提交，减少等待与复制，在 30 FPS 采集配置下实现一小时平均 29.83 FPS。

3）以完整帧事务连接检测与告警，区分目标离开和结果中断，通过配套上位机能清晰地观察目标。

1.6 设计流程

设计先明确监测需求与系统方案，再完成数据和模型准备、软硬件实现、联调优化，最后通过实板测试和现场观察验收。

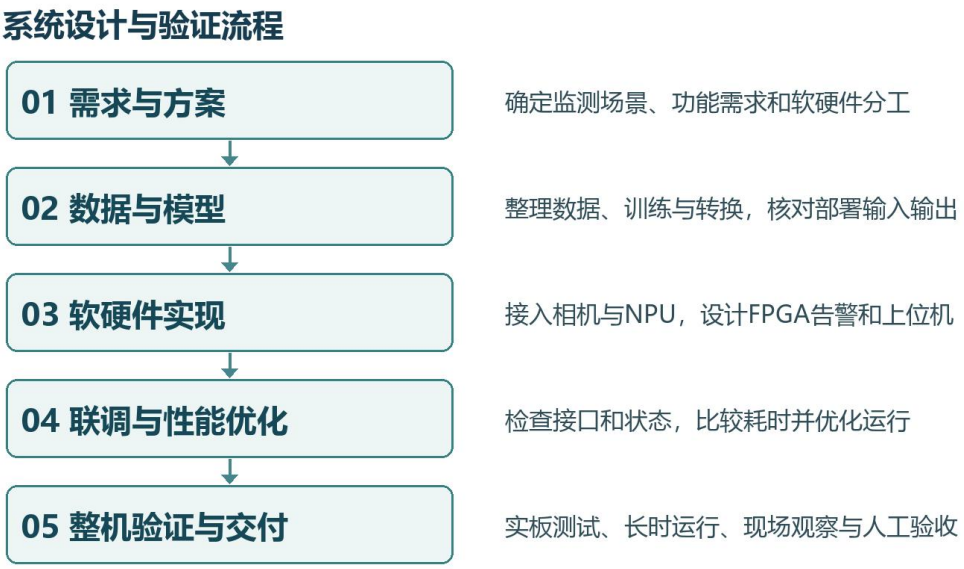


图 2　设计与验证流程

第二部分　系统组成及功能说明

2.1 整体介绍

系统由板端检测与告警、Windows 上位机两部分组成。图 3 展示从相机采集到实体反馈的数据流：CPU 取得最新图像并整理输入，NPU 完成模型计算，CPU 解码检测框并检查结果，再将原始框和帧号提交 FPGA。FPGA 按关注区域和连续有效帧更新告警，CPU 同时绘制画面、读取硬件状态并发布给上位机。

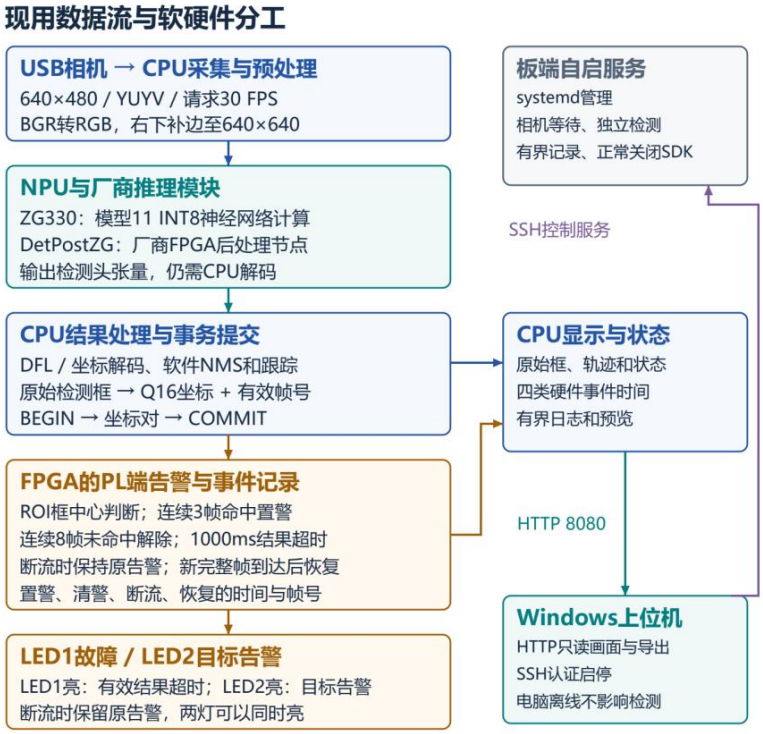


图 3　系统数据流与 CPU、NPU、FPGA 分工

这条数据流把计算任务和运行管理分开：模型只在板端加载一次，告警状态由硬件持续维护，预览与录像通过独立队列处理。表 1 列出各组成的任务。

代码包中 src/board/runtime/worker.py 连接采集、推理和告警，src/npu/product_stream.cpp 负责常驻 NPU 调用。

表 1　硬件组成及任务

组成	本作品中的任务
CPU	采集、预处理、框解码/NMS、跟踪、通信与服务管理
ZG330 NPU	运行固定模型 11 的 INT8 神经网络
厂商 FPGA 的 PL 端 DetPostZG	执行现用编译图中的检测后处理节点

组成	本作品中的任务
FPGA 的 PL 端	帧事务、ROI、连续确认、断流监督、事件记录与两路 LED
USB / SD / 网口	图像输入 / 开机启动 / 画面查看与认证控制

2.2 硬件系统介绍

2.2.1 硬件整体介绍

平台采用复旦微 FMQL30TAI 悟净开发板完整版[9]。CPU、ZG330 NPU 与 FPGA 的 PL 端集成于同一芯片，分别承担通用处理、神经网络计算和自定义逻辑。板端运行 Linux，USB 连接 Intel RealSense D435i 相机的彩色图像接口，SD 卡提供统一启动，有线网络连接上位机。

平台的供电、存储和外设接口沿用厂商设计。项目利用 USB 相机输入，在厂商 NPU 平台上部署模型，并新增区域告警、独立超时监督和硬件事件记录。

2.2.2 机械设计介绍

原型使用现有开发板与固定相机，通过 USB、供电线和网线连接，使用时固定视角并整理线缆；具体接线见图 13。

2.2.3 电路及逻辑模块介绍

FPGA 接收 CPU 提交的检测结果，先核验帧事务，再依次完成区域判断和连续帧确认。超时监督、LED 指示和事件记录独立配合，主要流程见图 4，模块职责见表 2。

对应代码位于代码包 src/pl/alarm/，其中 region_alarm_regs_m505.v 处理帧事务，region_alarm_core.v 维护连续确认与告警。



图 4　FPGA 的 PL 端模块与关键输入输出

表 2　告警逻辑模块

模块	关键输入	主要功能与输出
事务寄存器层	帧号、成对框坐标、BEGIN 与 COMMIT	检查完整性、框合法性和 ROI；返回状态及计数
告警规则核	commit、fresh、region_hit、frame_id	维护 alarm、fault、连续计数和事件
100 MHz 时基	业务时钟与业务复位	产生 1 ms tick 供断流计时
业务复位同步	业务复位请求、业务时钟	同步业务复位，与传输复位分开
事件观察模块	events、last_frame、100MHz 时钟	记录四类事件的最近周期数与帧号

帧事务与区域告警

CPU 通过 BEGIN、坐标写入和 COMMIT 提交一帧结果。硬件核对帧号、事务完整性及框范围后更新状态，每帧最多接收 16 个 Q16 归一化框。任一有效框的中心进入 ROI 即记为命中；连续 3 个有效帧命中后置警，连续 8 个有效帧无命中后解除。告警使用完整检测帧计数，目标跟踪编号用于显示。

超时、双灯与事件记录

连续 1000 ms 没有有效结果时，硬件置 fault、清除连续计数并保留已有告警；新的完整结果到达后恢复判断。LED2 显示目标告警，LED1 显示结果超时，两灯可同时亮。正常停止后，LED1 在约 1 秒后显示结果中断。

事件模块使用 100MHz 业务时钟，分别保存置警、清警、断流和恢复的最近一次周期数与帧号。软件读回后换算为从业务复位开始的相对时间，便于核查状态变化。

保留扩展中的像素通路

FPGA 的 PL 端预处理实现了固定取样、BGR 转 RGB 和补边，并完成共享 DDR 访问与数据交接。模式 A 将 1920×1080 按固定 3:1 中心取样成 640×360，模式 B 保持 640×480 有效图像，均在底部补值 114 至 640×640，见图 5。

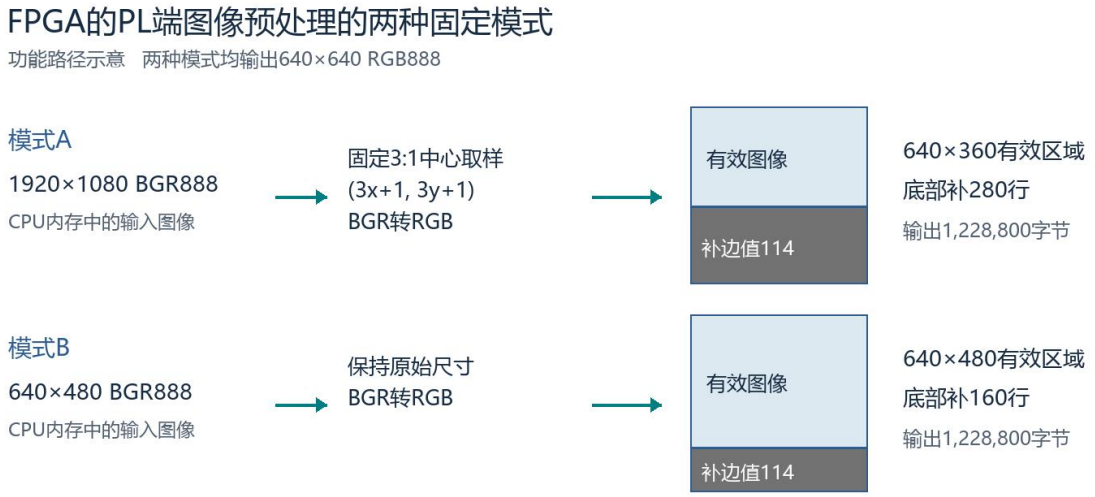


图 5　FPGA 的 PL 端预处理固定输入与补边方式示意

2.3 软件系统介绍

2.3.1 软件整体介绍

板端软件由采集、常驻推理、业务主循环和查看服务组成。V4L2 与 OpenCV 取得相机图像，C++进程通过 Icraft 调用 ZG330，Python 主循环完成输入校验、结果整理、FPGA 提交和状态发布。systemd 管理检测自启及配套服务，Windows 程序通过网络提供统一操作界面。

2.3.2 软件各模块介绍

数据与模型准备

模型以轻量化 YOLO11 为起点[1]，沿用开源网络结构。项目完成公开数据整理、单类别训练、导出、INT8 量化与实板部署，冻结训练实验 standard_visiodect_adapt_11 对应的候选作为“模型 11”，使权重、部署文件与实测记录保持对应。

模型冻结之前，先确定各数据来源在训练和开发中的用途。表 3 按实际训练清单统计 Anti-UAV、Det-Fly、Halmstad、DUT Anti-UAV、ICG-Drone 和 VisioDECT 的采用量[2-7]。低照 RGB 仍是可见光图像，与后文单独验证的热 IR 分支分开保存。

表 3　模型 11 使用的训练子集及来源

来源	去重图像	每轮曝光	重复	出处
Anti-UAV 日间 RGB	3421	3421	1	[2]
Anti-UAV 低照 RGB	335	335	1	[2]
Det-Fly	137	548	4	[3]

来源	去重图像	每轮曝光	重复	出处
Halmstad 可见光片段	373	1492	4	[4]
DUT Anti-UAV	2468	4936	2	[5]
ICG-Drone 室外子集	158	948	6	[6]
VisioDECT 选定机型	40	480	12	[7]
合计	6932	12160	—	—

表 3 中的 6932 为去重训练图像数，12160 为调整来源权重后的每轮清单曝光数。Halmstad 采用已有 TRAIN 视频片段，ICG 采用选定室外 TRAIN 成员，VisioDECT 采用作者版本 1。训练与开发分别保留图像、原标签、来源、视频组、帧号和用途清单，并核对文件摘要与解码像素重复关系。

开发集共 1629 张，包括 Anti-UAV 日间 1067 张、低照 96 张和 DUT 466 张，用于调试、选模及部署核验。公共数据原有标签沿用并注明原作者来源；团队检查核心数据的来源与用途，并对关键验证图进行人工核验。

模型导出检测头后，通过 Icraft 生成部署文件；INT8 校准从训练集选取 200 张图像。

采集与输入整理

采集模块交付最新完整帧，CPU 统一颜色、尺寸与补边方式。当前 640×480 相机图像由 BGR 转 RGB，底部以 114 补至 640×640。通用尺寸按比例缩放后补边，训练、电脑验证和板端部署采用一致的输入约定。

常驻推理与结果整理

C++常驻进程加载一次模型，连续接收图像并返回检测框和阶段计时。模型图包含厂商提供的 DetPostZG 检测后处理节点[10]，CPU 继续完成坐标解码与 NMS，置信度阈值为 0.25，NMS IoU 阈值为 0.7。相邻帧目标由跟踪器分配编号；素材展示版另绘制短轨迹，相机实测版采用框和编号显示。

业务主循环与告警提交

CPU 在 NPU 计算期间并行完成输入校验，收到结果后检查输入身份、帧龄与框范围，再解码并提交 FPGA。每次提交后比较 CPU 参考与硬件状态；遇到未知或不一致时停止处理并保留记录。图 6 给出取图、推理、提交和停止的运行顺序。

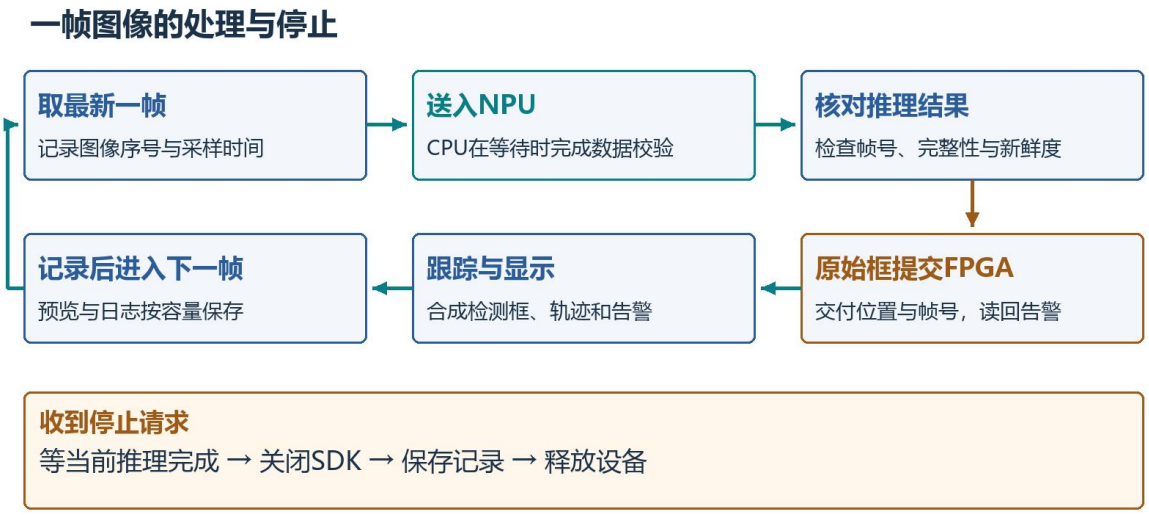


图 6　检测主循环和停止路径

自启、持续运行与正常停止

检测入口由 systemd 统一管理，通电后校验运行文件、等待相机就绪、创建会话并启动常驻推理进程。设备锁协调 NPU 使用，状态文件记录启动、检测和停止阶段。预览只保留最新帧，编码队列容量为 2，录像上限为 1200 帧；逐帧日志轮转，历史会话数量受限。

收到正常停止请求后，程序完成当前推理，关闭 SDK、结束编码与子进程并释放设备锁。达到录像上限后，检测和预览继续运行。

上位机与部署

自制 C# WinForms 上位机集中显示画面、FPS、运行时间和告警，侧栏提供连接设置、告警记录与使用说明。画面和状态通过 HTTP 读取，启停通过 SSH 密钥认证；暂停预览或关闭窗口后，板端检测继续。网线支持直连电脑或接入路由器。

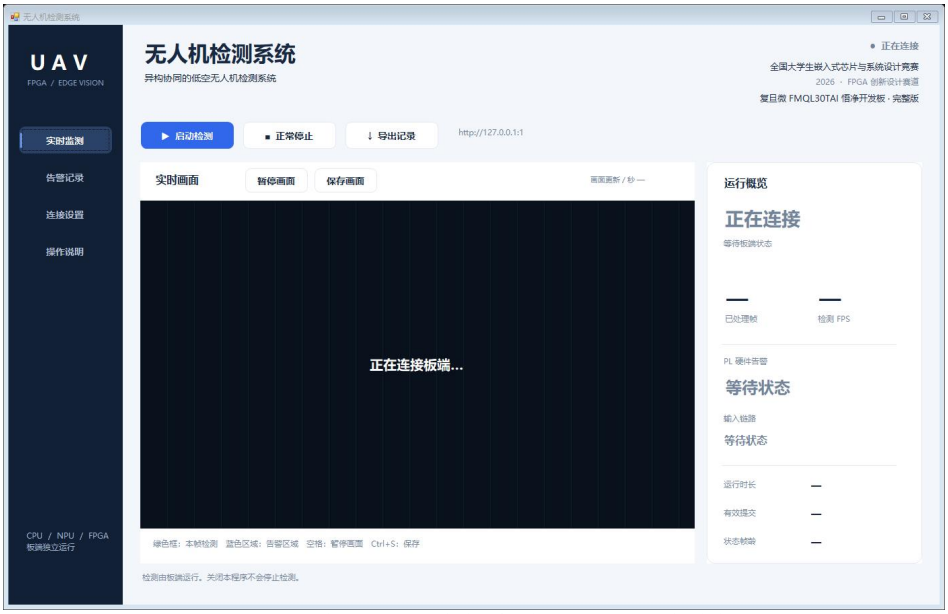


图 7　V7 上位机界面　本机离线布局及等待连接状态

提交材料包含源码、构建入口、运行程序、模型和实测记录，数据集注明来源与采用量。重新部署需要匹配的厂商 Linux 系统、Icraft 3.39、ZG330 运行库及配套 FPGA 平台文件，详细步骤保存在部署说明中。

2.3.3 团队与 AI agent 协作开发

项目按图 8 的流程推进。人工与 AI 讨论方案后，由 GPT/Codex 承担具体代码、脚本和材料初稿，Opus / Claude Code 作为独立技术复核参与。不同 Agent 通过共享仓库交接任务、源码与工程文件、复核意见及修订结果，使每次讨论落到可检查的文件。

下表按各环节内部工作量作粗略估计，由团队与 Agent 共同估算，只能大概表现不同任务的人工与 AI Agent 工具的分工与占比。在当前时代，我认为诚实地说明 AI 的贡献不是一件坏事。团队和两个 Agent 一致认为，单从工作量和成品来看，AI 深度参与了项目，其总的贡献占比在 60%以上。

表 4　人工与 AI 工具参与环节及工作量估计

环节	AI 工具参与	人工占比	AI 占比
方案讨论	GPT / Codex 讨论、整理	约 50%	约 50%
代码与工程	GPT / Codex 实现；Opus / Claude Code 复核	约 20%	约 80%
人工抽样与标签判断	Agent 提供 HTML 核验工具	约 70%	约 30%
实物操作与现场验收	Agent 辅助准备、记录与整理	约 90%	约 10%
测试分析与调试	GPT / Codex 脚本与记录分析	约 40%	约 60%
说明材料整理	GPT / Codex 整理草稿	约 40%	约 60%



图 8　方案讨论、实施复核、关键核查与人工验收

人工在模型选择、数据用途、代码与工程变更等关键节点介入核查，确定作品方向，并完成接线和现场操作。

数据检查中，团队审查训练集和开发集的来源、用途及重复关系，对关键验证图检查目标、修正标签并保存审核决定；图 9 是在一次运行过程中，人工审查发现 AI Agent 工具做任务期间发生了数据泄露，但其并未意识到，于是让 Agent 制作了一份 HTML 文件，便于人工复查。图中是一次关于验证集的 72 帧核验，此核验发现了 AI 工具漏掉的无人机并进行了人工补充。



图 9　人工数据核验界面　原图 坐标与审核决定

形成作品初稿后，人工结合运行记录检查功能、界面及说明材料，反馈修改并验收。以下三幅图展示一次技术复核、方案讨论和人工阅读代码的具体方式。

图 10 记录了对硬件配置流程的独立复核：Opus 结合 RTL 指出配置写入也受硬件门控约束，补充了此前只关注缓冲区释放条件时遗漏的问题，并给出调整控制流程的建议。

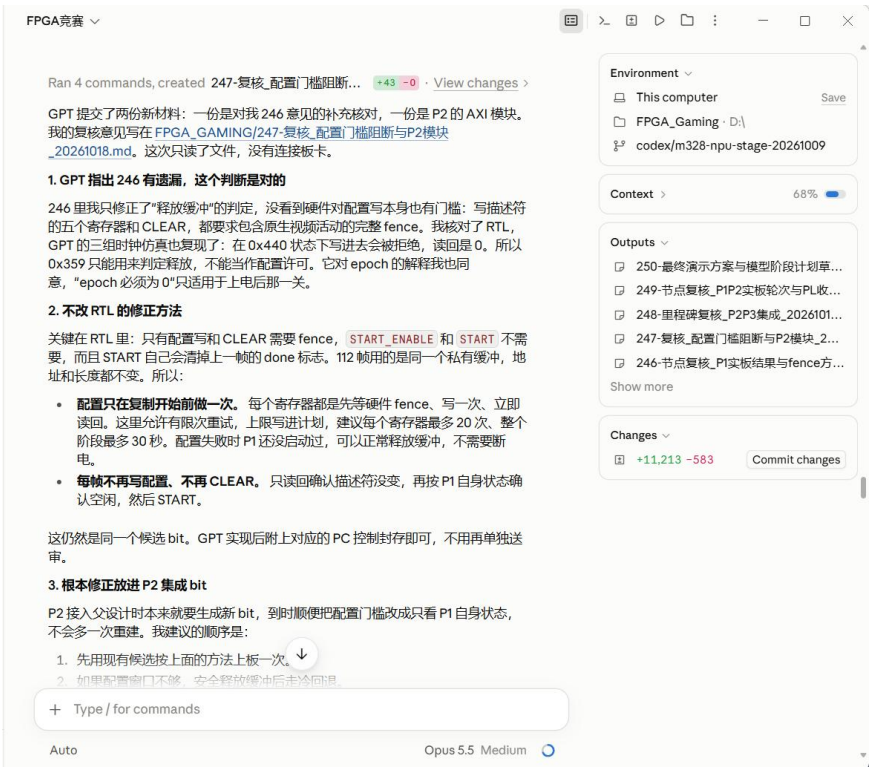


图 10　独立复核与问题修正

图 11 记录了团队提出应用场景和鸟类干扰问题后的讨论。针对教师—学生蒸馏，讨论提出同时检查鸟类误报、小目标召回和普通背景表现，为后续方案取舍提供参考。



图 11　团队提出方向后的方案讨论

图 12 是用于辅助人工阅读的离线代码页面。团队成员可以按模块查找文件和函数；图中以 worker.py 的 Hardware.commit 为例，引导核查帧号、目标框提交及 CPU 与 FPGA 状态对照，便于更快速的查看和核对 AI 工作的产出。

该页面随 GitHub 材料以“代码辅助阅读.html”提供；llm_usage.md 同时列出协作说明及证据入口。

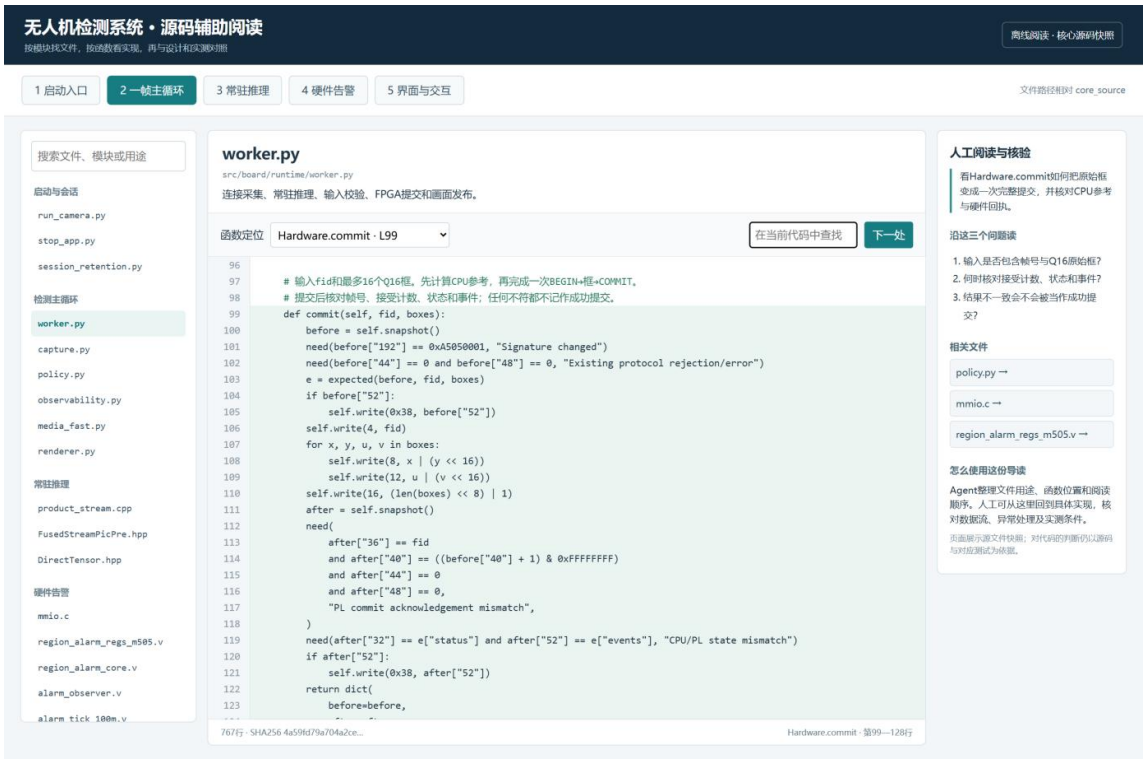


图 12　离线代码阅读页面：文件搜索、函数定位与核验提示

第三部分　完成情况及性能参数

3.1 整体介绍

当前交付版本以模型 11、CPU 预处理、NPU 检测和 FPGA 告警构成 RGB 相机整机。完成情况包括实物连接、板端程序、Windows 上位机和可追溯实测材料。下文先展示工程成果，再集中给出检测精度、告警响应、运行性能和扩展验证结果。

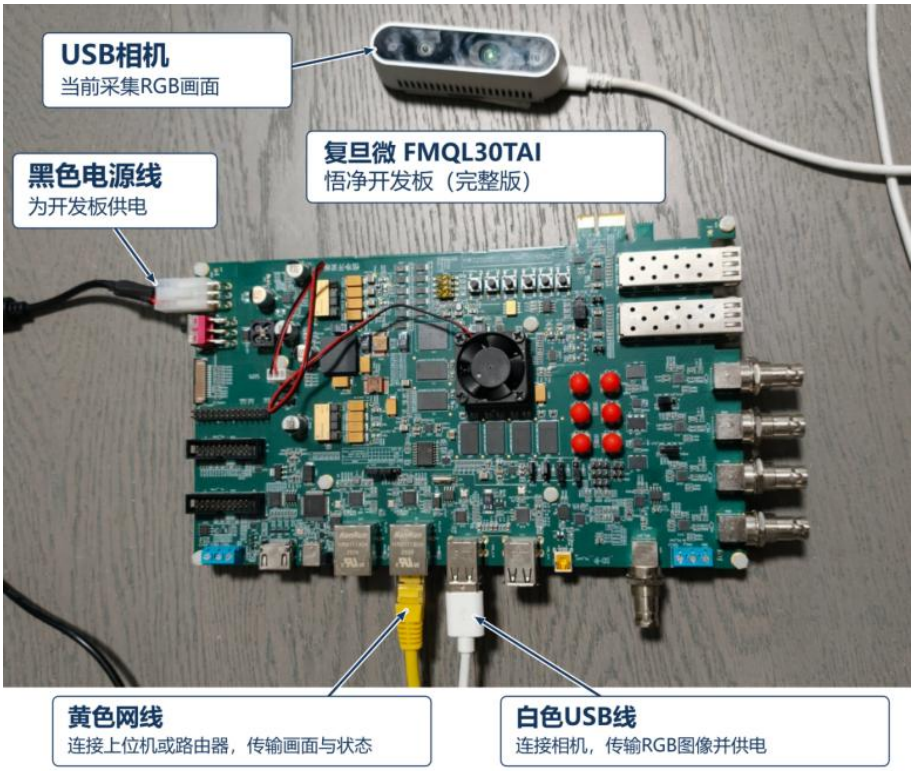


图 13　原型正面及主要接线

3.2 工程成果

3.2.1 机械成果

使用现有开发板与固定相机完成原型连接，整理供电与通信线缆。

3.2.2 电路成果

自研告警、独立业务时基、故障灯和事件记录已集成上板。实体 LED2 完成“灭—亮—灭”的目标告警实拍，LED1 故障状态也完成观察和记录。

3.2.3 软件成果

已形成板端持续检测程序、自启与正常停止入口、Windows 上位机，以及模型部署文件和源码构建入口。上位机完成实时显示、认证启停、画面保存和记录导出，界面及操作功能见 2.3.2 节图 7；第二台电脑的迁移与操作检验通过。

3.2.4 保留扩展的完成情况

独立 IR 模型完成训练、INT8 量化及固定图像实板推理；FPGA 像素通路完成两种固定输入的取样、颜色转换、补边与 DDR 交接验证。当前 RGB 整机采用 CPU 预处理，两项扩展的结果集中列于 3.3.7 节。

3.3 特性成果及量化测试

3.3.1 模型检测精度与部署一致性

在 1629 张开发图像、1609 个标注目标上，对模型 11 进行 FP32 与实板 INT8 对照。该开发集已用于调试和选模；评价固定置信度 0.25、NMS IoU 0.7，并以 IoU≥0.5 匹配目标框。表 5 汇总结果。

表 5　已曝光开发集上的 FP32 与 INT8 结果

统计量	FP32 参考	实板 INT8
图像数 / 标注目标	1629 / 1609	1629 / 1609
TP / FP / FN	1546 / 82 / 63	1541 / 70 / 68
精确率 P	94.96%	95.65%
召回率 R	96.08%	95.77%
F1	95.52%	95.71%

实板 INT8 精确率为 95.65%、召回率为 95.77%、F1 为 95.71%。Host INT8 与 NPU 均输出 1611 个框，逐框匹配，验证了量化模型的部署一致性。分来源检查中，DUT 子集 466 张图像的精确率为 89.07%、召回率为 89.44%，后续将重点补充这类复杂背景样本。

3.3.2 告警一致性、连续确认与故障响应

在同一板卡上，用-O3 编译的 C 参考与 FPGA 处理相同目标框、帧号和规则，766 次有效提交的状态、计数和事件一致，四组断流恢复通过。暂停 CPU 比较进程约 1.22 秒时，硬件仍按独立时基报告断流。

按约 30 个有效检测帧/秒估算，连续 3 帧命中时，第三帧在首帧后约 67 ms；连续 8 帧无命中时，第八帧在首个无命中帧后约 233 ms。已有片段对照中，加长确认时间增加了置警延迟，成品采用 3 帧置警、8 帧解除的配置。

新硬件版本完成真实冷启动与 90 秒相机检测，2647 次推理和提交一致。置警、清警、断流及恢复四类事件定向核对通过，结果超时约 999 ms；测试结束 SDK 正常关闭。事件记录保存各类最近一次的业务时钟周期与帧号。

3.3.3 运行速度与优化对照

推理调度优化

对连续NPU算子采用SDK合并提交后，同一组40张TRAIN图像的前向均值由24.36 ms降至11.24 ms，原始检测框和分数逐字节一致；反向复测原配置为23.99 ms。计时包含搬运、计算和同步等待，优化位于SDK调用层，两组使用相同FPGA逻辑。

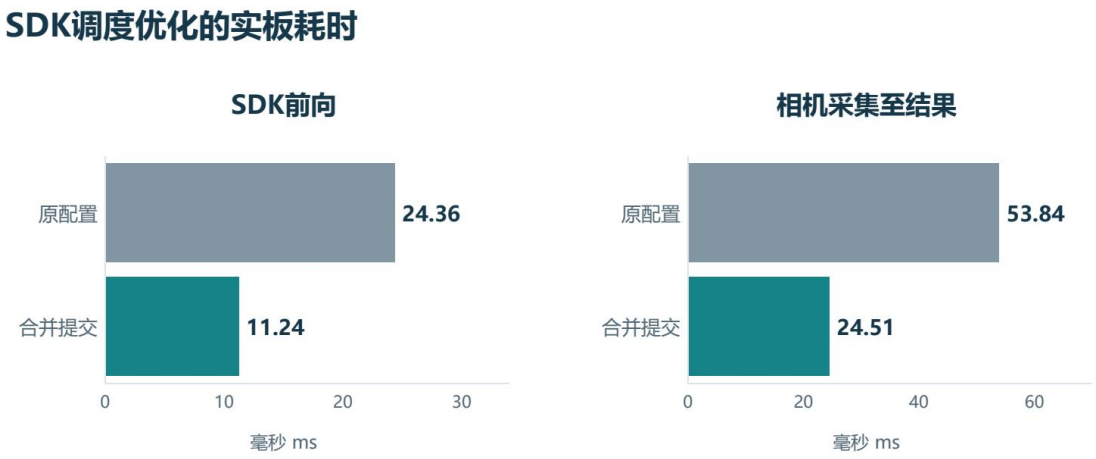


图 14　SDK 调度优化的实板耗时对照

两段各20秒的相机短测排除首60帧后，处理速度由约26.14提升至29.98 FPS，采集至结果的平均耗时由53.84降至24.51 ms。随后30秒回归完成859次推理与FPGA提交，未知状态为0，并正常关闭SDK。

CPU与FPGA接口优化

在模型调用之外，我们还减少了CPU读取FPGA状态的接口开销。原来每轮读取7个寄存器，需要7次Python到C的跨语言调用；改为一次C接口后，仍按相同顺序读取相同寄存器，不改变硬件规则。排除首60帧后的相机样本中，状态读取及告警提交段由1.510降至0.824 ms / 帧，减少45.4%，平均每帧节省约0.686 ms。

表6使用相同目标框、帧号和规则，每组统计128次，分别测量C规则计算、独立进程往返和FPGA提交读回。

表 6　告警处理开销　单位μs　各组 128 次

框数	C 规则计算 中位/P99	C 独立进程往返 中位/P99	FPGA 的 PL 端提交读回中位 /P99
0	0.250 / 0.518	41.655 / 56.306	41.085 / 46.524
1	0.280 / 0.427	41.920 / 46.497	52.540 / 57.305
4	0.310 / 0.568	42.190 / 47.581	86.365 / 103.244
16	0.430 / 0.555	42.300 / 44.839	221.195 / 236.046

C 规则计算中位耗时低于 1 μs；FPGA 一列包含软件提交与状态读回，中位耗时约 0.04—0.22 ms。系统将告警状态和超时监督放在 FPGA，主要利用其独立时基和持续维护状态的能力。

3.3.4 启动、停止与一小时稳定性测试

系统已保留真实通电自启与新硬件冷启动记录。当前完整版本通过既有服务入口执行一小时相机长测，使用 640×480、30 FPS 采集配置，保持预览和有界录像；模型加载与首帧等待另计。表 7 按完整主循环 3600.122 秒统计平均处理帧率。

本次长测摘要见 GitHub 材料 05_evidence/camera_1h_final_result.json，其他指标及对应范围见同目录实测结果.md。

表 7　当前完整版本的一小时相机长测

项目	实测结果
时长 / NPU / FPGA 的 PL 端	3600.122 s / 107396 次 / 107396 次
整段处理帧率	29.831 FPS；相机请求 30 FPS
结果完整性	未知 0；107396 次推理，107396 次有效提交
CPU 与 FPGA 的 PL 端对照	主循环逐次核对，留存尾部 5760 行；错误即停止
Python 中位 RSS	首末 5 分钟 76620→78300 KiB，变化 1.641 MiB
NPU 中位 RSS	首末 5 分钟 36812→34704 KiB，变化-2.059 MiB
退出与录像	SDK 正常关闭；录像 1200 帧，编码队列丢弃 0

全程完成 107396 次推理与 107396 次有效提交，平均 29.831 FPS，未知状态为 0；正常停止后 SDK 关闭。预览约 6.76 张/秒，录像保留 1200 帧，编码队列丢弃为 0。逐次提交核对持续启用，轮转日志最终保留尾部 5760 行。

3.3.5 FPGA 资源占用与时序结果

当前路由报告中，自研告警、事件记录与时基合计 270 LUT、830 FF，另有复位同步器 1 LUT、2 FF；DSP 和 BRAM 均为 0。

表 8　当前路由检查点的告警、事件记录与时基资源

模块	LUT / FF	内部 Setup 裕量	内部 Hold 裕量
business_alarm	252 / 811	5.157 ns	0.141 ns
business_tick	18 / 19	8.083 ns	0.123 ns
合计	270 / 830	工作时钟 100 MHz	仅模块内部路径

包含厂商框架的完整父工程，Setup / Hold 余量分别为+0.030 / +0.029 ns，最小周期 / 脉宽检查为−0.409 ns。

功耗方面，当前缺少便于获取整机输入功率的测量条件，本次暂未测量；后续可在供电输入端记录电压和电流，补充运行状态下的功率。

3.3.6 实板演示与现场测试

选取未参与本模型训练和开发的无人机视频，在板端使用模型 11 进行推理。图 15 展示了视频约第 78.3 秒的检测结果，右侧为同一帧的局部放大；所选 75—78.8 秒片段的 114 帧均有检测框，反映出模型在新视频场景下具有一定泛化能力。



图 15　未参与训练和开发的视频场景：模型 11 的 NPU 实板检测结果

2026 年 10 月 30 日，团队在宿舍外飞行大疆无人机，室内放置板卡、相机与上位机，完成傍晚楼宇场景检测，实况见图 16。本次现场画面用于定性观察，没有做逐帧标注。但能够在较暗的、复杂楼宇背景的情况下检出较小的无人机目标，已经能说明我们模型的能力是过关的。

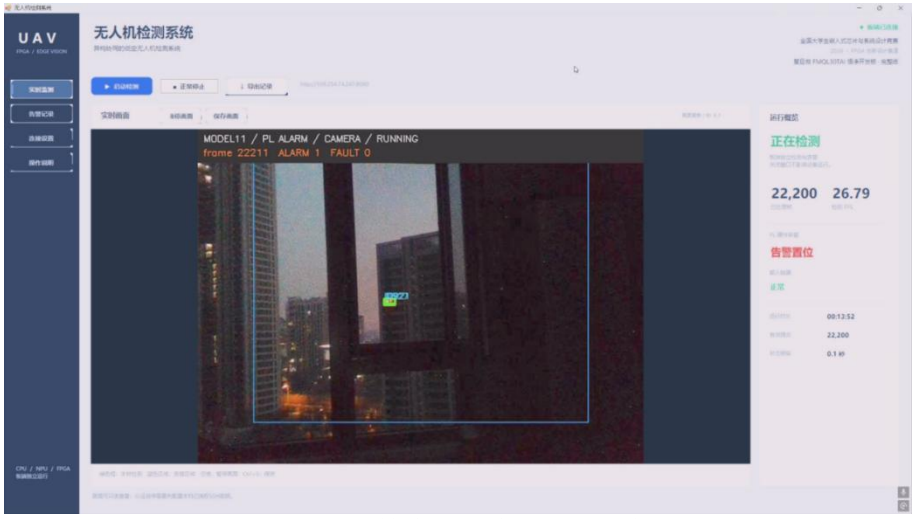


图 16　日落后楼宇场景实测　上位机画面

3.3.7 保留扩展的验证结果

IR 模型实板结果

考虑到夜间光线不足时，可见光图像细节和目标对比度会下降，因此我们还做了红外 IR 扩展工作，但由于缺乏相关设备，因此只做了初步验证，工作不够全面，仅作为扩展内容展示。

IR 扩展采用 640×512 输入，上下各补 64 行至 640×640。在 608 张 IR 开发图像上，Host 量化仿真与 NPU 实板的 552 个输出框逐框匹配。图 17 展示输出对照，表 9 汇总该分支精度；IR 与 RGB 分别使用各自的数据、权重及统计口径。

该结果使用已参与开发的 IR 图像，反映固定图像上的部署表现；后续若有机会可以补充外部来源和真实夜间场景测试。

为看清检测结果，测试保留了目标与负帧的完整统计：608 帧中共有 564 个标注目标和 44 帧负例，实板结果为 542 个正确检出、10 个误检、22 个漏检。IR 模型与当前 RGB 模型分别管理，现有结果主要用于检验红外模型的量化、部署和固定图像推理。

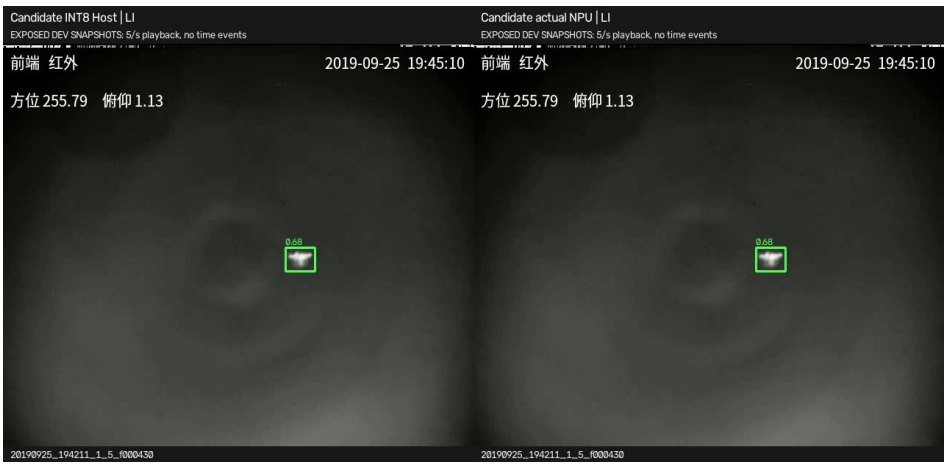


图 17　IR 模型的电脑量化仿真与 NPU 实板结果

表 9　独立 IR 候选在固定开发图像上的实板结果

固定开发图像	精确率 P	召回率 R	F1
608 张	98.19%	96.10%	97.13%

FPGA 图像预处理结果

两种固定输入均完成 CPU 参考与 FPGA 输出逐字节比较，后续 20 帧配对测试结果一致。耗时测试使用相同输入和同一 ARM 核，每种模式预热 2 对后统计 8 对；FPGA 全通路计时包括复制、缓存交接和完成等待，结果见表 10。

像素通路将输入图像写入共享缓冲区，由 PL 完成固定取样、颜色通道转换和补边，再将结果交回 CPU。两种输入分别对应 1080p 的固定 3:1 取样和 640×480 原尺寸处理，输出均整理为 640×640；对照时逐字节比较输出，而不是只观察画面。

表 10　像素预处理 CPU 与 FPGA 的 PL 端全通路耗时　均值

固定输入模式	CPU 平均耗时	FPGA 的 PL 端全通路平均耗时
1920×1080 固定 3:1 取样	2.282 ms	19.962 ms
640×480 通道转换与补边	0.648 ms	11.243 ms

两种模式下 CPU 耗时分别为 2.282 ms 和 0.648 ms，FPGA 全通路为 19.962 ms 和 11.243 ms。该扩展完成像素功能验证，速度较慢我们怀疑可能是任务较轻，数据搬运的开销很大，因此当前系统选用耗时较低的 CPU 预处理方案。

第四部分　总结

4.1 可扩展之处

除了上文提到的保留扩展工作 IR 模型和 FPGA 图像预处理值得进一步细化和深入以外，后续可补充扩展的内容还有很多：

1、可以尝试做不同时段、天气、距离及鸟类背景的独立测试数据，在冻结阈值后评价泛化；

2、可以尝试鸟类负例与教师学生蒸馏等算法，来优化模型的抗干扰能力，这点我们初步做了，但是会导致小目标召回率降低，因此舍弃了；

3、还可以尝试结合变焦、云台、多机位或可见光与红外融合，改善小目标观察。

另外，工程方面还需完善机壳、供电与远程健康监测，补做功耗和更长时间运行测试等；FPGA 在本项目中深度参与，但并没有完全体现其加速收益，这一点也可以进一步探讨。

4.2 心得体会

首先，前期我们把较多精力放在目标能否被检出，对数据划分和验收条件准备得较晚。开发集多次参与调试与选模，后来我人工检查发现了数据泄露问题，让 AI 制作了如图 9 的 HTML 文件辅助我核验，这让我和团队深刻意识到了 AI Agent 只能作为辅助，我们必须在项目的每一步都深度参与，才能在使用 AI 工具提升效率的同时交付一个优秀的作品。

然后是关于 FPGA 的 PL 端在此项目中出现的问题。我们花了大量时间做 FPGA 预处理，也确实实现了功能，但搬运与交接耗时显著高于 CPU。后来发现，我们在开始预处理方案前就应该对这件事有预估，我们不具备足够的前瞻性因此导致花费了大量精力在一个未能成为最终成品的部分上。后来的图像操作合并、校验与推理并行、SDK 提交优化，我们都吸取了这一经验，最终通过实板对照取得收益。这个问题使我们认识到，设计阶段就应统一输入和计时范围，再决定哪些计算放到哪一侧。

从文件演示走向相机整机，设备使用协调、相机等待、正常停止、日志轮转与录像上限都影响实际体验。完成一个作品和整理、交付一个作品是完全不同的，如果能更早的意识到这一点，尽早借到无人机，解决供电问题，找到一个更利于展示的场景多拍几组实板测试，会让我们的模型更具说服力，由于 10 月底才开始准备，现场视频主要集中在傍晚楼宇背景。补充新视频的板端检测后，我们也认识到，仍应在不同距离、照度和背景下积累更完整的验证素材。

本项目是团队和 AI Agent 协作完成的异构协同的低空无人机检测系统，在最后我们要特别感谢复旦微和赛事平台能给我们团队这次机会，我们在这一个多月获得了非常多的成长。

第五部分　参考文献

[1] JOCHER G, QIU J. Ultralytics YOLO11[CP/OL]. 2024[2026-10-29]. https://github.com/ultralytics/ultralytics

[2] JIANG N, WANG K, PENG X, et al. Anti-UAV: A Large Multi-Modal Benchmark for UAV Tracking[EB/OL]. arXiv:2101.08466, 2021. 原作者数据与代码：https://github.com/ucas-vg/Anti-UAV

[3] ZHENG Y, CHEN Z, LV D, et al. Air-to-Air Visual Detection of Micro-UAVs: An Experimental Evaluation of Deep Learning[J]. IEEE Robotics and Automation Letters, 2021, 6(2): 1020-1027. Det-Fly 数据：https://github.com/Jake-WU/Det-Fly

[4] SVANSTRÖM F, ALONSO-FERNANDEZ F, ENGLUND C. A dataset for multi-sensor drone detection[J]. Data in Brief, 2021, 39: 107521. DOI:10.1016/j.dib.2021.107521. 原作者数据：https://github.com/DroneDetectionThesis/Drone-detection-dataset

[5] ZHAO J, ZHANG J, LI D, WANG D. Vision-based Anti-UAV Detection and Tracking[EB/OL]. arXiv:2205.10851, 2022. DUT Anti-UAV 数据：https://github.com/wangdongdut/DUT-Anti-UAV

[6] XIAO J, CHEE J H, FEROSKHAN M. Real-Time Multi-Drone Detection and Tracking for Pursuit-Evasion With Parameter Search[J/OL]. IEEE Transactions on Intelligent Vehicles, 2024. DOI:10.1109/TIV.2024.3360433. ICG-Drone 数据：https://github.com/NTU-ICG/multidrone-detection-tracking

[7] AJAKWE S O. VisioDECT for Scenario-Based Multi-Drone Detection[DB/OL]. Version 1, 2025[2026-10-29]. 作者公开数据版本 DOI：https://doi.org/10.34740/kaggle/dsv/13155551

[8] OpenCV. Open Source Computer Vision Library[CP/OL]. [2026-10-29]. https://github.com/opencv/opencv

[9] 上海复旦微电子集团股份有限公司. 悟净开发板硬件使用手册[Z]. 厂商提供资料。

[10] 复旦微. FPAI DEMO 说明[Z]. 厂商提供的参考设计资料。